

EXAM LAB

SUBMITTED BY:

RABEEA FATIMA

REGISTRATION NO: 2023-BSE-049

SECTION: B

SUBJECT: CLOUD COMPUTING

SUBMITTED TO:

SIR M.SHOAIB

QUESTION 1:

AWS IAM Setup Using AWS CLI and Console Verification

1. Create IAM group

```
@RabeeaFatima14 /workspaces/LabExam (main) $ aws iam create-group --group-name SoftwareEngineering
{
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPAXHUW46YMTVIZQW2GS",
    "Arn": "arn:aws:iam::497458935321:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:33:42+00:00"
  }
}
```

2. View IAM group details

```
@RabeeaFatima14 /workspaces/LabExam (main) $ aws iam get-group --group-name SoftwareEngineering
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPAXHUW46YMTVIZQW2GS",
    "Arn": "arn:aws:iam::497458935321:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:33:42+00:00"
  }
}
```

3. Create IAM user

```
@RabeeaFatima14 /workspaces/LabExam (main) $ aws iam create-user --user-name RabeeaFatima
{
  "User": {
    "Path": "/",
    "UserName": "RabeeaFatima",
    "UserId": "AIDAXHUW46YM6NMBANYXO",
    "Arn": "arn:aws:iam::497458935321:user/RabeeaFatima",
    "CreateDate": "2026-01-19T07:35:44+00:00"
  }
}
```

4. View IAM user details

```
@RabeeaFatima14 /workspaces/LabExam (main) $ aws iam get-user --user-name RabeeaFatima
{
  "User": {
    "Path": "/",
    "UserName": "RabeeaFatima",
    "UserId": "AIDAXHUW46YM6NMBANYXO",
    "Arn": "arn:aws:iam::497458935321:user/RabeeaFatima",
    "CreateDate": "2026-01-19T07:35:44+00:00"
  }
}
```

5. Add the IAM user to the SoftwareEngineering group

```
@RabeeaFatima14 /workspaces/LabExam (main) $ aws iam add-user-to-group --user-name RabeeaFatima --group-name SoftwareEngineering
@RabeeaFatima14 /workspaces/LabExam (main) $ aws iam get-group --group-name SoftwareEngineering
{
  "Users": [
    {
      "Path": "/",
      "UserName": "RabeeaFatima",
      "UserId": "AIDAXHUW46YM6NMBANYXO",
      "Arn": "arn:aws:iam::497458935321:user/RabeeaFatima",
      "CreateDate": "2026-01-19T07:35:44+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPAXHUW46YMTVIZQW2GS",
    "Arn": "arn:aws:iam::497458935321:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:33:42+00:00"
  }
}
```

6. Attach AdministratorAccess managed policy to the SoftwareEngineering group

```
@RabeeaFatima14 /workspaces/LabExam (main) $ aws iam list-policies --query 'Policies[?PolicyName==`AdministratorAccess`]'
[
  {
    "PolicyName": "AdministratorAccess",
    "PolicyId": "ANPAIWMBCKSKIEE64ZLYK",
    "Arn": "arn:aws:iam::aws:policy/AdministratorAccess",
    "Path": "/",
    "DefaultVersionId": "v1",
    "AttachmentCount": 1,
    "PermissionsBoundaryUsageCount": 0,
    "IsAttachable": true,
    "CreateDate": "2015-02-06T18:39:46+00:00",
    "UpdateDate": "2015-02-06T18:39:46+00:00"
  }
]
```

7. View the admin access

```
@RabeeaFatima14 [ ] /workspaces/LabExam (main) $ aws iam attach-group-policy --group-name SoftwareEngineering --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
@RabeeaFatima14 [ ] /workspaces/LabExam (main) $ aws iam list-attached-group-policies --group-name SoftwareEngineering
{
  "AttachedPolicies": [
    {
      "PolicyName": "AdministratorAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
    }
  ]
}
```

8. Verify in the AWS console

Users (2) [Info](#) ⌂ Delete Create user

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

☐	User name	Path	Group:	Last activity	MFA	Password age	Console last sign-in	Acc
☐	Rabeea	/	0	6 minutes ago	-	23 days	16 days ago	Act
☐	RabeeaFatima	/	1	-	-	-	-	-

User groups (1) [Info](#) ⌂ Delete Create group

A user group is a collection of IAM users. Use groups to specify permissions for a collection of users.

☐	Group name	Users	Permissions	Creation time
☐	SoftwareEngineering	1	Defined	10 minutes ago

QUESTION 2:

Terraform Lab: Simple AWS Environment with Nginx over HTTPS

1. Configure the AWS provider

```
@RabeeaFatima14 [ ] /workspaces/LabExam/terraform-infra (main) $ cat provider.tf
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "me-central-1"
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
  profile = "default"
}
```

2. Configure input variables

```
@RabeeaFatima14 [ ] /workspaces/LabExam/terraform-infra (main) $ cat variables.tf
variable "vpc_cidr_block" {
  description = "CIDR block for VPC"
  type        = string
}

variable "subnet_cidr_block" {
  description = "CIDR block for subnet"
  type        = string
}

variable "availability_zone" {
  description = "Availability zone for resources"
  type        = string
}

variable "env_prefix" {
  description = "Environment prefix for resource naming"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}
```

3. Create VPC and subnet

```
@RabeeaFatima14 [ ] /workspaces/LabExam/terraform-infra (main) $ cat main.tf
# VPC
resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block

  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

# Subnet
resource "aws_subnet" "myapp_subnet" {
  vpc_id            = aws_vpc.myapp_vpc.id
  cidr_block        = var.subnet_cidr_block
  availability_zone = var.availability_zone

  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}
```

4. Create Internet Gateway and configure default route table

```

# Internet Gateway
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id

  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

# Configure default route table
resource "aws_default_route_table" "myapp_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

```

5. Discover public IP and compute /32 CIDR using data + locals

```

# Get current public IP
data "http" "my_public_ip" {
  url = "https://icanhazip.com"
}

# Compute /32 CIDR from public IP
locals {
  my_ip = "${chomp(data.http.my_public_ip.response_body)}/32"
}

```

6. Configure the default security group in the VPC

```

# Configure default security group
resource "aws_default_security_group" "myapp_sg" {
  vpc_id = aws_vpc.myapp_vpc.id

  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [local.my_ip]
  }

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTPS from anywhere"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

```

7. Create an AWS key pair for SSH

```
@RabeeaFatima14 @ /workspaces/LabExam/terraform-infra (main) $ if [ ! -f ~/.ssh/id_ed25519 ]; then
> ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
> fi
Generating public/private ed25519 key pair.
Your identification has been saved in /home/codespace/.ssh/id_ed25519
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:1Nft2t3daeQ24DkiwhYK/zzySb4WaFiB7VYxSZflpvc codespace@codespaces-9bea99
The key's randomart image is:
+--[ED25519 256]--+
|      . . . . o . |
|      = . o . . |
|      . o + . o . |
|      o o o . o . |
|      .. =.S . o . |
|      .o+o+. . oo*. =|
|      oo.=... ..+.E=|
|      .*oo . . + . |
|      +Bo          |
+-----[SHA256]-----+
```

8. Create the EC2 instance resource

```
# EC2 Instance
resource "aws_instance" "myapp_server" {
  ami           = "ami-0a2b9ad578ecee9c5"
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet.id
  vpc_security_group_ids = [aws_default_security_group.myapp_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name        = aws_key_pair.myapp_key.key_name
  user_data       = file("entry-script.sh")

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
```

9. Create entry-script.sh to configure Nginx + HTTPS

```
GNU nano 7.2 entry-script.sh
#!/bin/bash

# Update system
dnf update -y

# Install Nginx
dnf install -y nginx

# Create self-signed certificate
mkdir -p /etc/nginx/ssl
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/nginx.key \
  -out /etc/nginx/ssl/nginx.crt \
  -subj "/C=PK/ST=Islamabad/L=Islamabad/O=MyOrg/CN=localhost"

# Create custom HTML page
cat > /usr/share/nginx/html/index.html << 'HTML'
<!DOCTYPE html>
<html>
<head>
  <title>Rabeea's Terraform Environment</title>
</head>
<body>
  <h1>Welcome to Rabeea's Terraform Environment</h1>
  <p>This infrastructure was deployed using Terraform.</p>
  <p>Server is running Nginx with HTTPS enabled.</p>
```

```

GNU nano 7.2
server_name _;

ssl_certificate /etc/nginx/ssl/nginx.crt;
ssl_certificate_key /etc/nginx/ssl/nginx.key;

location / {
    root /usr/share/nginx/html;
    index index.html;
}

server {
    listen 80;
    server_name _;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
NGINXCONF

# Enable and start Nginx
systemctl enable nginx
systemctl start nginx

```

10. Add Terraform output for public IP

```

GNU nano 7.2
output "ec2_public_ip" {
    description = "Public IP address of the EC2 instance"
    value       = aws_instance.myapp_server.public_ip
}

```

11. Add Terraform output for public IP

```

GNU nano 7.2 terraform.tfvars
vpc_cidr_block      = "10.0.0.0/16"
subnet_cidr_block   = "10.0.10.0/24"
availability_zone    = "me-central-1a"
env_prefix           = "dev"
instance_type        = "t3.micro"

```

12. Run Terraform commands and capture outputs

```
@RabeeaFatima14 [ ] /workspaces/LabExam/terraform-infra (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Finding latest version of hashicorp/http...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Terraform plan

```
Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ec2_public_ip = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run
"terraform apply" now.
```

Terraform apply

```
+ network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ec2_public_ip = (known after apply)
aws_instance.myapp_server: Creating...
aws_instance.myapp_server: Still creating... [00m10s elapsed]
aws_instance.myapp_server: Creation complete after 14s [id=i-085b6698c76c09236]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Terraform output

```
@RabeeaFatima14 [ ] /workspaces/LabExam/terraform-infra (main) $ terraform output
ec2_public_ip = "3.28.226.187"
```

13. Verify Terraform resources in AWS console

Your VPCs

VPCs | VPC encryption controls

Your VPCs (1/2) Info Last updated less than a minute ago Actions Create VPC

Find VPCs by attribute or tag

	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv
☐	dev-vpc	vpc-01b80a6d5e22536aa	Available	-	-	Off	10.
☒	-	vpc-0ae3cbbba6356c8c4e	Available	-	-	Off	17.

vpc-0ae3cbbba6356c8c4e Settings Down Arrow

Details | Resource map | CIDRs | Flow logs | Tags | Integrations

Details

VPC ID
vpc-0ae3cbbba6356c8c4e

DNS resolution
Enabled

Main network ACL
acl-0a9af23da67a1059e

State
Available

Tenancy
default

Default VPC
Yes

Block Public Access
Off

DHCP option set
dopt-0d62cc61ce78583b0

IPv4 CIDR
172.31.0.0/16

DNS hostnames
Enabled

Main route table
rtb-014f248916c3e3363

IPv6 pool
-

Actions Create internet gateway

Internet gateways (2) Info Last updated 1 minute ago Actions Create route table

Find internet gateways by attribute or tag

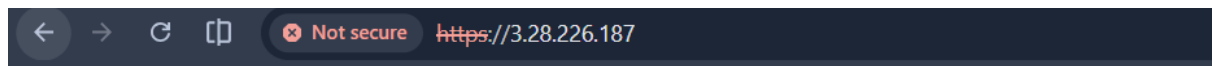
	Name	Internet gateway ID	State	VPC ID	Owner
☐	-	igw-03386bd28d48a30b8	Attached	vpc-0ae3cbbba6356c8c4e	497458935321
☐	dev-igw	igw-09c69dab4e65bcae8	Attached	vpc-01b80a6d5e22536aa dev-vpc	497458935321

Route tables (2) Info Last updated 1 minute ago Actions Create route table

Find route tables by attribute or tag

	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	dev-rt	rtb-0afe880f5490cf7c7	-	-	Yes	vpc-01b80a6d5e22536aa dev-
☐	-	rtb-014f248916c3e3363	-	-	Yes	vpc-0ae3cbbba6356c8c4e

14. Verify HTTPS access from the browser



Welcome to Rabeea's Terraform Environment

This infrastructure was deployed using Terraform.

Server is running Nginx with HTTPS enabled.

QUESTION 3:

Ansible Playbook for EC2 Web Server Using Q2 Instance

1. Create Ansible inventory file hosts

```
GNU nano 7.2 hosts *
[ec2]
40.172.186.199

[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

2. Create project-level ansible.cfg

```
GNU nano 7.2 ansible.cfg
[defaults]
host_key_checking = False
inventory = ./hosts
interpreter_python = /usr/bin/python3

[privilege_escalation]
become = True
become_method = sudo
become_user = root
```

3. Create Ansible playbook

```
GNU nano 7.2 my-playbook.yml
---
- name: Configure EC2 instance with Apache HTTPD
  hosts: ec2
  become: true
  tasks:
    - name: Update all system packages
      ansible.builtin.dnf:
        name: "*"
        state: latest
        update_cache: yes

    - name: Check if nginx is installed
      ansible.builtin.command: rpm -q nginx
      register: nginx_check
      ignore_errors: yes
      changed_when: false

    - name: Stop nginx service if running
      ansible.builtin.systemd:
        name: nginx
        state: stopped
        enabled: no
        when: nginx_check.rc == 0
        ignore_errors: yes

    - name: Uninstall nginx package if present
      ansible.builtin.dnf:
        name: nginx
```

4. Run the Ansible playbook

```
@RabeeaFatima14 [ ] /workspaces/LabExam/ansible (main) $ ansible ec2 -m ping
40.172.186.199 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

```

TASK [Fetch public hostname using IMDSv2] *****
ok: [40.172.186.199]

TASK [Print public IP address] *****
ok: [40.172.186.199] => {
  "msg": "Instance Public IP: 40.172.186.199"
}

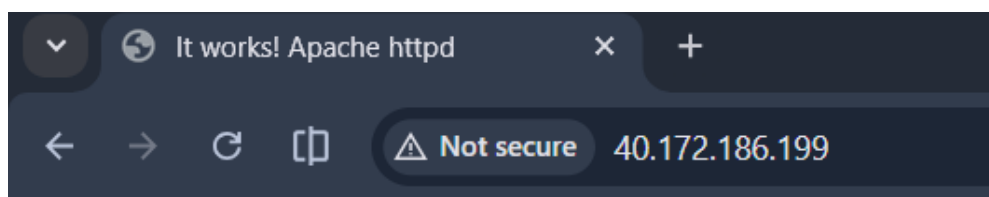
TASK [Print public hostname] *****
ok: [40.172.186.199] => {
  "msg": "Instance Public Hostname: "
}

TASK [Restart httpd service] *****
changed: [40.172.186.199]

PLAY RECAP *****
40.172.186.199      : ok=11  changed=3  unreachable=0  failed=0  skipped=2  rescued=0  ignored=1

```

5. Verify HTTP access



It works!

CLEANUP

Terraform destroy

```

aws_instance.myapp_server: Still destroying... [id=i-03325d4fcdffb049f8, 01m00s elapsed]
aws_instance.myapp_server: Still destroying... [id=i-03325d4fcdffb049f8, 01m10s elapsed]
aws_instance.myapp_server: Destruction complete after 1m13s
aws_key_pair.myapp_key: Destroying... [id=serverkey]
aws_subnet.myapp_subnet: Destroying... [id=subnet-03725ba475f860e4d]
aws_default_security_group.myapp_sg: Destroying... [id=sg-0d88a5ff03daf9941]
aws_default_security_group.myapp_sg: Destruction complete after 0s
aws_key_pair.myapp_key: Destruction complete after 1s
aws_subnet.myapp_subnet: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-027609f664663ed17]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.

```

Instances are terminated

Instances (2) Info								
Find Instance by attribute or tag (case-sensitive)					All states	< 1 > ⚙		
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☐	dev-ec2-insta...	i-085b6698c76c09236	Terminated	t3.micro	-	View alarms +	me-central-1a	-
☐	dev-ec2-insta...	i-03325d4fcdffb049f8	Terminated	t3.micro	-	View alarms +	me-central-1a	-

User group and user is deleted

User groups (0)

Info

A user group is a collection of IAM users. Use groups to specify permissions for a collection of users.

Q Search

< 1 > ⚙

Group name

▲

Users

▼

Permissions

▼

Creation time

No resources to display

Users (1)

Info

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Q Search

< 1 > ⚙

User name

▲

Path

▼

Group:

▼

Last activity

▼

MFA

▼

Password age

▼

Console last sign-in

Acc

Rabeea

/

0

✔

7 minutes ago

-

✔

23 days

✔

16 days ago

Act